



NAIJAGO PLATFORM

Developer guide

Verified against the deployed build at naijago.netlify.app. What to click on day one, the nine rules that cost money or trust if you get them wrong, a twelve-phase build order with acceptance criteria, the API that has to sit behind it, and the bugs already hit in this codebase.

VERSION

2.0

DATE

4 September 2026

RECONCILED TO

Live v1.4.1

COMPANION

Platform map

Contents

Twelve sections. Sections 4, 5 and 7 carry most of the weight: the rules that cost money, the build order with acceptance criteria, and the bugs already hit in this codebase.

01 What you have

02 Day one

03 How the code is laid out

04 Nine rules that will bite you

05 Build order, with acceptance criteria

06 The API you need behind it

07 Gotchas already hit in this codebase

08 Testing

09 Vocabulary

10 Copy that is load bearing

11 Still to decide

12 Definition of done

Reconciled against the deployed build at `naijago.netlify.app` (v1.4.1).

The deployed site is the source of truth. The repository at `github.com/justinlagos/naijago` is byte-identical to it: 21 of 24 files match on SHA-256, and the three that differ are Netlify's injected hosting meta in `index.html`, plus `netlify.toml` and `package.json`, which are build config and are not published. Where this guide and the code disagree, the code wins and this guide needs a patch.

Read this first. The other documents are reference; this one is the map.

01 What you have

A complete frontend of NaijaGo. Every screen, every state, every flow, wired and clickable. No backend, no database, no persistence. It is an installable PWA with an offline app shell.

Treat it as a specification you can run. It answers questions a Figma file cannot: what the hold timer does at three minutes, what the bank timeout screen says, what happens to a waitlist place when the claim window lapses, what the gate shows the door staff.

Take the CSS and the behaviour. Replace the JavaScript. It is deliberately framework free with a hash router so it opens in any browser with no build step. That is not a recommendation for production. The stylesheets are production shaped, tokens then primitives then components then a responsive layer loaded last, and they port to any stack unchanged.

What is real: the layout, the tokens, the copy, the states, the arithmetic, the accessibility work, the icon family, the ad geometry and rate card. What is fixture: the eight experiences, the three hosts, the photography, the demo ad creatives, the reference booking.

02 Day one

Run it

```
python3 -m http.server 8000
```

Then open `http://localhost:8000`. The service worker needs an origin, so `file://` will not do.

Click these seven things before you read further

WHERE	WHAT TO NOTICE
Tap any heart while signed out	The auth wall records what you were doing and replays it. Build this before checkout.
<code>#/experience/beach-rave</code> , quantity 2	$30,000 + 1,500 = 31,500$. Two fees, one visible.
Sit on <code>#/checkout</code>	At three minutes the hold strip turns amber. At zero you get the expiry screen.
The three simulate buttons on checkout	Four failure screens. Read the bank timeout one twice.
<code>#/pass/NG-8842-L0S</code>	Four pass states and an offline toggle on chips at the bottom.
<code>#/calendar</code> , change a filter	The month grid repaints, not just the list. That is the point of a calendar.
<code>#/advertise</code>	The whole revenue product: inventory, creative, owner review, payment, live, reporting.

Press `A` on any screen to run the contrast audit. It returns zero on the deployed build; keep it that way.

Read order

1. This guide
2. `SPEC.md` and `ARCHITECTURE.md` for the fixture canon, routes and flows
3. `DESIGN-SYSTEM.md` before you write any CSS
4. `UI-AUDIT.md` for the icon family and the corrections applied in v1.3
5. `IMPLEMENTATION.md` for what has to be server side
6. `ADVERTISING.md` and `CALENDAR.md` when you reach those surfaces
7. `IMAGES.md` before anything ships to a real user

The platform map covers all of it in one drawing. Put it on a second monitor.

03 How the code is laid out

index.html	shell, plus the inline service-worker registration
manifest.webmanifest	PWA manifest
sw.js	app-shell cache, naijago-shell-v1.4.1
netlify.toml	publish config, not served
scripts/build.mjs	build helper
css/	
tokens.css	every colour, type and geometry token
base.css	reset, type scale, layout primitives, controls
site.css	chrome
home.css	homepage sections, plus 19 ad selectors
app.css	product screens, calendar, plus 37 ad selectors
responsive.css	1100px and 760px, then the 44px touch target floor
js/	
data.js	the fixture canon, 26 exported constants
util.js	in-memory state, helpers, the QR drawing, cart maths
components.js	render fragments, the icon family, adSlot, nativeAdCard
view-home.js	nine sections, hero gallery, story rail
view-discover.js	explore, seasons, guides, hosts
view-calendar.js	the one calendar
view-advertise.js	the self-serve advertising product
view-book.js	detail, auth wall, checkout, rails, failures
view-pass.js	pass states, gate, waitlist, transfer, reschedule
view-account.js	eight account routes
view-partner.js	five partner console routes
app.js	router, hold timer, section reveals, editorial controls
audit.js	the WCAG contrast sweep
test/e2e.mjs	Playwright assertions

Keep the CSS load order. Tokens, base, site, home, app, responsive. The responsive file is last because the touch target floor inside it has to beat every earlier rule. If you move it, targets silently drop below 44px on mobile and nothing tells you.

Keep the token names. `--gold-*`, not `--brass-*`. They match the deployed build.

Bin the router, the view functions and the delegated event handler. They exist so the prototype runs without a build step.

04 Nine rules that will bite you

1. Two fees. Never conflate them. The buyer pays 5% on top of face value. The host pays 8% out of face value and keeps 92%. Different numbers, different jobs, separate ledger entries. Do not create a single "platform fee" column; you will lose the ability to answer a host's question about their own statement.

```
buyer_total = face + round(face * 0.05)
host_payout = face - round(face * 0.08)
points      = round(buyer_total * 0.01)    // pending until scanned in
```

2. Money is integer kobo. Never floats. Round once, at the point of charge, and store the rounded figure. Do not recompute a displayed total from a stored face value at render time or the receipt and the charge will eventually disagree by one naira.

3. No card data touches your code. Card entry goes in the processor's hosted field. Your own form never sees a PAN, a CVV or an expiry.

4. The hold is inventory, not a UI timer. Ten minutes, server owned, atomic. Two people clicking the last two seats at once must not both succeed. Releasing a hold raises availability: release two seats from a listing showing seven left and it shows nine. That was inverted in an early draft and it read perfectly fluently, which is exactly why it survived a review.

5. The four failure states ship with the success path. Declined, bank timeout, underpaid, expired. Not a follow-up ticket. The timeout state is the one that matters: you do not know whether the money left the account, so the screen says so and tells the user not to pay twice. Every payment attempt carries an idempotency key for the same reason.

6. The pass is cached at issue, not at open. "98.4% first-try scans" is a published promise. It holds only if the pass payload and its assets are cached before the user reaches the venue, and if the gate resolves from its own manifest downloaded before doors. `sw.js` caches the app shell today. The pass payload is not in it yet, and that is the gap between the promise and the build.

7. Keep advertising out of the paid path, and enforce it. Today nothing calls `adSlot` from the booking or fulfilment views, so the exclusion holds by convention. There is no `NG.AD_FREE` in the deployed build. Add the guard before the ad server goes in. See `ADVERTISING.md` section 6.

8. Meta text uses `--n-600`, never `--n-500`. `--n-500` is 2.57:1 on parchment and is for placeholders and decorative fills only. Both tokens are live and correct today; keep them that way.

9. Focus outlines stay. `2px solid var(--gold-500)` at `outline-offset: 3px`, on every interactive element, on every surface.

05 Build order, with acceptance criteria

Twelve phases. Each is shippable and each has a test you can point at.

Phase 1: tokens and primitives.

Port `tokens.css`, `base.css` and the six shared components.

Done when: the contrast audit passes on a page containing one of each, in both themes, and every control clears 44px under a coarse pointer.

Phase 2: the icon family.

The geometric set from v1.1, applied through `NG.icon` and `NG.vibeIcon`.

Done when: no generic clipped-square markers remain. See `UI-AUDIT.md`.

Phase 3: chrome.

Ticker, sticky header, footer, five-action mobile dock, search dialog, event dialog, toast.

Done when: the dialog traps focus, closes on Escape and returns focus to its trigger.

Phase 4: homepage.

Nine sections, hero gallery with manual controls, story rail, progressive section reveals.

Done when: the six vibe counts sum to 184, which is what the "184 in Lagos" link claims. If those two disagree, the page is lying. And every reveal respects `prefers-reduced-motion`.

Phase 5: explore and detail.

Faceting, sort, the empty state, mobile filters in a bottom sheet so results start in the first viewport.

Done when: on a 390px viewport, organic results are visible without scrolling past a filter stack.

Phase 6: the auth wall.

Records `{ kind, id, back }`, replays on success.

Done when: tapping a heart while signed out, signing in, and landing back on the same screen with the item saved. Do not start phase 7 until this passes.

Phase 7: checkout, hold, rails, failures.

All together.

Done when: two clients racing for the last seat produce one hold and one clear rejection, and a replayed payment request charges once.

Phase 8: pass, offline, gate.

Signed pass payload, cache at issue, gate manifest, scan write.

Done when: the pass renders with the network disabled and the gate admits from a cached manifest with the network disabled.

Phase 9: the calendar.

Month grid, three filters, selected date.

Done when: filters repaint the grid and an empty day is legible.

Phase 10: account, waitlist, plans.

Done when: a lapsed offer moves to the next person and returns the holder to the back of the queue, not the front.

Phase 11: partner console.

Fewest users, most arithmetic.

Done when: every payout row is exactly 92% of its gross, or has an itemised refund against it explaining why not.

Phase 12: advertising.

The four placements, then `#/advertise` against a real ad server and a real payment rail.

Done when: no unit renders without a label, none renders in the paid path, and a campaign cannot go live without passing owner review.

06 The API you need behind it

A sketch, not a contract.

Auth

```
POST /auth/otp/request    { contact }
POST /auth/otp/verify     { contact, code }    → { token, user }
```

Catalogue

```
GET /listings?city&vibe&area&band&sort&cursor    → { items[], nextCursor }
GET /listings/:id
GET /calendar?month=2026-09&city&vibe&type       → { days: [{ date, count, ids[] }] }
GET /hosts/:id · /seasons · /guides
```

The calendar endpoint returns counts per day, not full listings. The grid needs 31 numbers, not 31 objects.

Booking

```
POST /holds              { listingId, tierId, qty }
                          → { holdId, expiresAt, totals: { face, fee, total } }
DELETE /holds/:id
POST /orders              { holdId, rail, idempotencyKey }
                          → { orderId, status, railPayload }
GET /orders/:id           → { status: pending|paid|declined|underpaid|expired }
POST /webhooks/payments/:provider (server to server, signature verified)
```

`totals` come back from the server. The client displays them; it never computes the number that gets charged.

Passes and the gate

```
GET /passes/:ref          → signed payload, cacheable
POST /passes/:ref/transfer { to }
POST /passes/:ref/reschedule { occurrenceId }
GET /gate/manifest/:occurrenceId → cached admit list, downloaded before doors
POST /gate/scan            { signedRef, doorId, scannedAt }
                          → { admit, reason }
```

Sign the reference. A screenshot of someone else's pass should fail a signature check, not merely a duplicate check.

Waitlist, account, host

```
POST /waitlist { listingId } · GET /waitlist/:listingId/position
POST /waitlist/offers/:id/claim          → creates a hold, returns it
GET /me · /me/bookings · /me/passes · /me/saved · /me/plans · /me/notifications
GET /host/listings · /host/payouts · /host/refunds
POST /host/listings · POST /host/listings/:id/submit
```

Advertising

```
GET /ads/inventory                      → the four formats and live rates
GET /ads/slot?format=&route=&city=&vibe= → creative, or 204 No Content
POST /ads/campaigns { name, format, headline, url, starts, ends, budget }
                                          → { id, status: 'review' }
POST /ads/campaigns/:id/approve         (owner only)
POST /ads/campaigns/:id/pay             { idempotencyKey }
GET /ads/campaigns/:id/report           → impressions, clicks, spend
```

204 matters. A slot that returns nothing must collapse, not leave a hole.

07 Gotchas already hit in this codebase

A `url()` inside a CSS custom property resolves against the stylesheet, not the document. Setting `style="--img:url(assets/x.jpg)"` inline and consuming it as `background-image: var(--img)` in a stylesheet requests `/css/assets/x.jpg`. Set `background-image` inline directly.

`position: sticky` creates a stacking context. A sibling that follows it in flow paints on top. A half-page unit below a sticky booking box intercepted every click on the checkout link. Make the column sticky, not the box.

`String.prototype.replace` treats `$$` in the replacement as a literal `$`. A bundler spliced the JS payload in as a replacement and silently turned `NG.$$` into `NG.$`, breaking the header at runtime. Use the function form when the replacement contains code.

A contrast audit that reads `backgroundCoLoR` without compositing lies in both directions.

`rgba(255,255,255,.14)` over ink is not white. Walk the ancestor chain and blend before measuring. `js/audit.js` does this.

Text over photography cannot be scored by the formula. Those containers carry `data-over-image` and the audit skips them. They are verified against the scrim spec: every image carrying text has a gradient reaching `rgba(10,10,9,.9)` at the text end.

Same-URL fragment navigation does not reload the page. Tests that assume state resets between routes will quietly carry a signed-in session across twenty assertions. Force a reload when the target URL matches the current one.

A service worker that caches the app shell will serve stale JS. `sw.js` is network-first with a cache fallback and the cache name carries the version, `naijago-shell-v1.4.1`. Bump it on every release or returning users get the previous build.

08 Testing

`test/e2e.mjs` serves the folder, drives a real Chromium and asserts on rendered output. Port these regardless of stack:

- the money arithmetic, on every screen it appears
- the auth wall replaying intent
- the waitlist lapse going to the back of the queue
- the vibe counts summing to the number the page claims
- every payout row being 92% of its gross
- zero contrast failures across every route
- no horizontal overflow at 390, 430, 768 and 1440
- no interactive element under 44px on mobile
- no ad unit unlabelled, and none in the paid path
- the service worker serving the shell with the network disabled

Run the contrast audit in CI. `js/audit.js` returns an array; fail the build when it is non-empty.

09 Vocabulary

Hosts, not vendors, partners or organisers. The exception is the header nav link and the section eyebrow, which say "Vendors" because the site does. **Experiences**, not events. **Passes**, not tickets. **Placements**, not ad units, in anything an advertiser reads.

Header navigation is Discover, Calendar, Festivals, Vendors, Guides. The mobile dock is Home, Explore, Calendar, Saved, Passes.

10 Copy that is load bearing

- The **bank timeout** message says we do not know whether the money left the account. It prevents a double payment. Do not make it more confident.
 - The gate says **one scan, then it stops working**, not "cannot be photographed twice". The second is an absolute the product cannot enforce.
 - **Refund ETAs quote the rail's real speed**. Cards are 5 to 10 working days because the card scheme says so.
 - The **clash message** names the actual obstacle: "That is a boat and a bridge in sixty minutes."
 - The advertising page promises **context first, human reviewed, clear reporting**. All three are checkable claims. Keep them true.
-

11 Still to decide

DECISION	BLOCKS
Enforce the paid-path ad exclusion in code	Ad server integration
What "platform tax" is on the payment step, and at what rate	Advertiser billing
Ad close controls and frequency capping	Ad server integration
Ad category exclusions (alcohol and betting are live here)	Sales
Caching the pass payload, not just the app shell	The 98.4% promise
Offline double-scan tie-break when two doors both admit	Gate reconciliation
Deep-linking a calendar day; more months; more cities	Calendar
Whether guide-only listings belong in the calendar	Product
Replacing the photography	Anything user facing. See <code>IMAGES.md</code>

12 Definition of done

Per pull request:

- ☐ Contrast audit returns an empty array on every route touched
- ☐ No interactive element under 44px at 390px width
- ☐ No horizontal overflow at 390, 430, 768, 1440
- ☐ Focus visible on every new interactive element
- ☐ Every new surface has its empty, loading and error state built
- ☐ Money displayed comes from the server, not recomputed client side
- ☐ New placements are labelled and absent from the paid path
- ☐ `sw.js` cache name bumped if any shell asset changed
- ☐ Copy matches section 10 where it overlaps
- ☐ `prefers-reduced-motion` respected on anything that moves